



ECOM SCHOOL

Urban Online Academy & Network

Penetration Test Report

Ecom College

August 4th 2026
Black Box Penetration Testing

This legal disclaimer regulates the usage of this Penetration Testing report and the content of this report. The contents of this document have been prepared from the information collected by the "ECOM" research team. It is worth noting that the information offered in this report constitutes a general framework for security screening and is not intended to ensure protection or compliance with any requirements and guidelines of any authority and the like. Nor can it be concluded that compliance with the recommendations of this report will ensure that the subject of the test is protected and that it does not contain any additional vulnerabilities. Under no circumstances will the Penetration Testing team and the " ECOM " team be responsible for any damage directly or indirectly caused by this test of any kind. The contents of this entire document are confidential and completely confidential and are the intellectual property of the " ECOM " Research Team and may be used only internally for educational purposes only. In addition, it is worth noting that the structure of this document is confidential and is the intellectual property of the " ECOM " research team and may not be used if you are not a member of this team.

TABLE OF CONTENT

REPORT STRUCTURE	3
ABOUT THE EDITOR	3
EXECUTIVE SUMMARY	4
BACKGROUND	4
PROJECT DESCRIPTION	4
SCOPE & TARGETS	4
TEST LIMITATIONS	5
SUMMARY & ASSESMENT	5
CONCLUSIONS	6
ATTACK TREE FOR COMPLEX SCENARIOS	6
SETTING GOALS AND OBJECTIVES	7
IDENTIFIED VULNERABILITIES	8
VULN-001 <VULN FULL NAME> (INFORMATIVE/LOW/MEDIUM/HIGH/CRITICAL)	8
FINDING DETAILS	9
VULN-<NO. IN 3 DIGITS> <VULN NAME>	9
CVSS	9
RISK	9
DESCRIPTION	9
PROOF OF CONCEPT	9
DETAILS	10
RECOMMENDED MITIGATIONS	10
APPENDICES	11
METHODOLOGY	11
APPLICATIVE PENETRATION TESTS	11
INFRASTRUCTURE PENETRATION TEST	13
FINDINGS CLASSIFICATIONS	15

REPORT STRUCTURE

This report contains three different sections:

1. **Executive Summary** - This section includes a brief description of the content of the work as well as a list of the main findings that constitute potential for damage and, as a result, require the organization to take corrective steps in our view.
2. **Details of the tests** - This section details all the tests performed by division into the various areas as well as a description of the information collected in the survey. This section also lists all the findings of the exam, the description of the risks as a result of the findings, and the recommendations for implementation based on the accumulated experience of ECOM.
3. **Appendices** - Brief of the methods used during the penetration test with additional explanation about our rating system fix effort.

ABOUT THE EDITOR

Shenhav Yehuda, Tomer Tal and Or Aviv, are students at Ecom School Cybersecurity.

Skilled in Computer Networking, Data Analysis, Computer Forensics, Reverse engineering, Infrastructure and Applicative Penetration Testing.

Shenhav Yehuda:



<https://www.linkedin.com/in/shenhav-yehuda-115738251/>

Tomer Tal:



<https://www.linkedin.com/in/tomer-tal-39ab013a8/>

Or Aviv:



<https://www.linkedin.com/in/or-aviv-7506a5391/>

EXECUTIVE SUMMARY

BACKGROUND

The students Shenhav, Tomer and Or of the Cyber Security Team was asked to perform an infrastructure + applicative penetration test for the Ecom College on August 4th 2026 for a CTF final task in Rishon LeZion, Israel.

CTF task targets: **(1)** Read a file named `/flag.txt`. **(2)** Gain higher privileges and take over the system. 7 vulnerabilities were found during the test.

This Penetration Test was conducted as part of the practical training and final project requirements of the Cyber Security course at ECOM College.

The purpose of the assessment was to provide hands-on experience in identifying, analyzing, and validating security vulnerabilities in a controlled environment while applying industry-standard penetration testing methodologies and techniques. The assessment simulated realistic attack scenarios to evaluate the security posture of the target environment and to demonstrate the student's ability to perform a structured security assessment.

This report documents the methodology used throughout the assessment, the vulnerabilities identified, supporting Proof of Concept (PoC) evidence, risk analysis, and recommended remediation actions. The report is intended solely for educational purposes and demonstrates the practical knowledge and technical skills acquired during the course. This report used black-box penetration testing.

All testing activities were performed under the supervision and guidelines of ECOM College as part of the Cyber Security curriculum. The findings presented in this report reflect the security status of the tested environment at the time the assessment was conducted and may no longer be applicable if changes have been made to the target systems after the completion of the assessment.

PROJECT DESCRIPTION

SCOPE & TARGETS

In advance with the client, the test team was given the following goals:

No.	Target Address	Extra Details
1	https://techie-world.net/	The target domain relays on http://98.94.89.209/ , mission targets: Read <code>/flag.txt</code> file and get higher privileges to take over the system for CTF task.

This test contains a number of infrastructural / applicative test methodologies in order to examine the level of risk of the information that is output in the identified systems.

As part of this examination, the following were examined:

- A number of code injection techniques at both the client and server level that can significantly compromise the information stored in this system.
- OWASP TOP 10 includes a variety of vulnerabilities and advanced attack techniques.
- Check for system bugs that can lead to malicious actions at the user level.
- Several techniques for scanning and finding known weaknesses in customer systems.

TEST LIMITATIONS

- **Time Constraints:** Due to a limited testing timeframe, certain potential vulnerabilities and attack vectors could not be fully investigated or verified.
- **Strict Scope Limitations:** The penetration testing activities were strictly confined to the domain <https://techie-world.net/>. Any other subdomains, systems, or connected infrastructure were explicitly out of scope and remained untouched.
- **Geographical & Team Dispersion:** The geographical distribution of the team members created coordination challenges and delayed the overall workflow, ultimately reducing the effective operational schedule.
- **No Social Engineering:** Including phishing, vishing, and physical tailgating against organization employees, were out of scope for this assessment.

SUMMARY & ASSESMENT

During the test it was found that an attacker can perform RCE using a web shell along with other misconfiguration defects.

The penetration testing lasted 7 days, and resulted several vulnerabilities such as RCE, LFI, Broken Access Control, etc. An attacker that is exploiting these bugs may gain full control on the system. This may damage the organization's reputation and may put the organization and its clients at risk.

Reconnaissance:

At the beginning of the assessment process, we conducted several preliminary tests to gather initial information about the target system, its environment, and general characteristics. As part of the reconnaissance phase, we performed subdomain enumeration and conducted Open-Source Intelligence (OSINT) activities to identify publicly available information that could support the subsequent stages of the assessment.

Part 1:

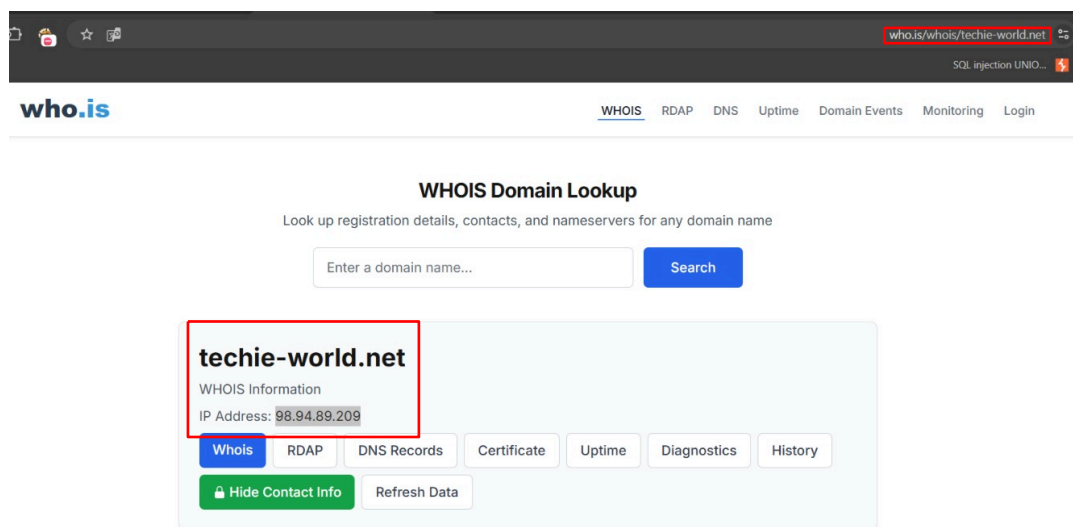
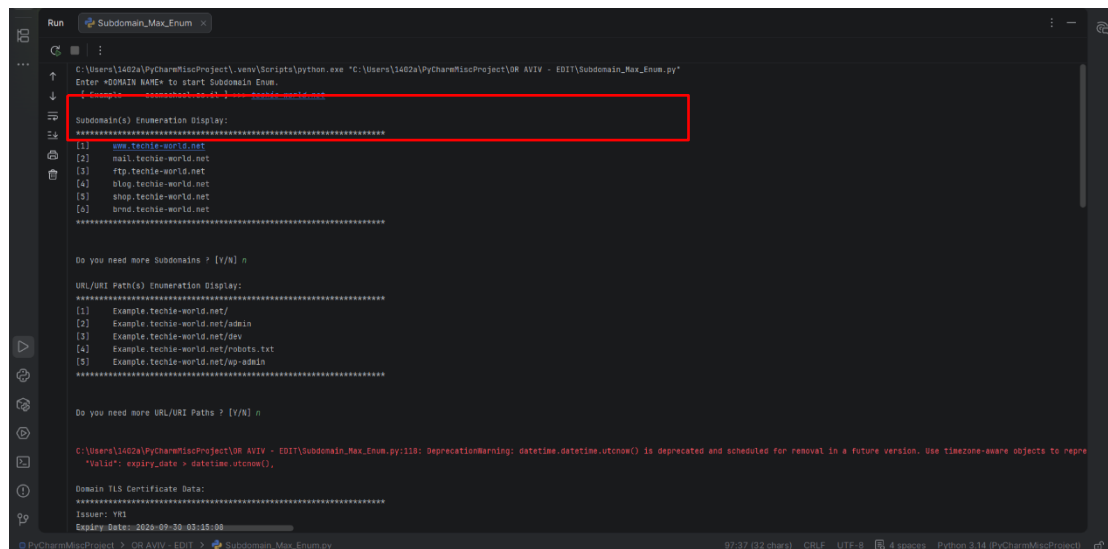


Figure 1 – OSINT via using who.is website.

Part 2:



```
Run Subdomain_Max_Enum x
C:\Users\1402a\PyCharmMiscProject\venv\Scripts\python.exe "C:\Users\1402a\PyCharmMiscProject\OR AVIV - EDIT\Subdomain_Max_Enum.py"
Enter *DOMAIN NAME* to start Subdomain Enum.
Domain: techie-world.net

Subdomain(s) Enumeration Display:
*****
[1] www.techie-world.net
[2] mail.techie-world.net
[3] ftp.techie-world.net
[4] blog.techie-world.net
[5] shop.techie-world.net
[6] brnd.techie-world.net
*****

Do you need more Subdomains ? [Y/N] n

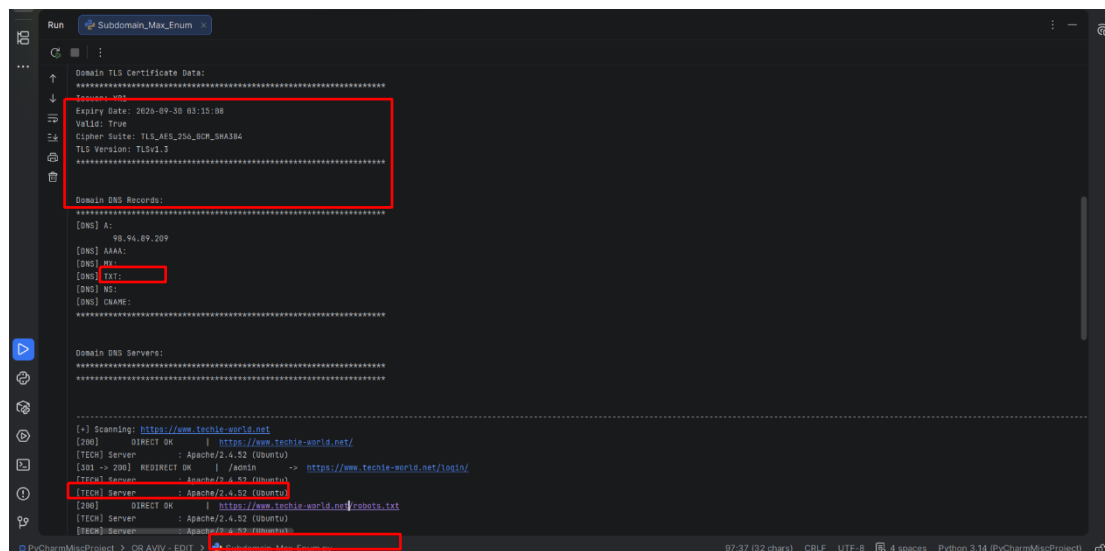
URL/URI Path(s) Enumeration Display:
*****
[1] Example.techie-world.net/
[2] Example.techie-world.net/admin
[3] Example.techie-world.net/dev
[4] Example.techie-world.net/robots.txt
[5] Example.techie-world.net/wp-admin
*****

Do you need more URL/URI Paths ? [Y/N] n

C:\Users\1402a\PyCharmMiscProject\OR AVIV - EDIT\Subdomain_Max_Enum.py:118: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to repre
"valid": expiry_date > datetime.utcnow(),

Domain TLS Certificate Data:
*****
Expiry Date: 2020-09-30 03:15:08
Issuer: VM1
Expiry Date: 2020-09-30 03:15:08
```

Figure 1 – Sub-domain enumeration script: Inserting the target domain name for testing.



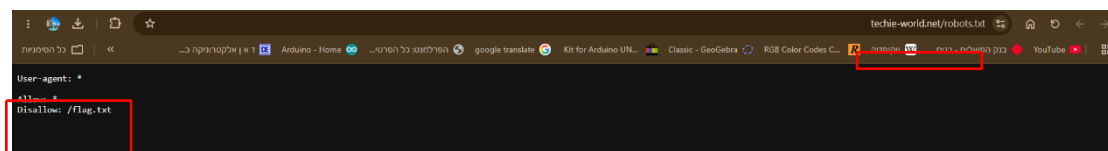
```
Run Subdomain_Max_Enum x
Domain TLS Certificate Data:
*****
Expiry Date: 2020-09-30 03:15:08
Valid: True
cipher suite: TLS_AES_256_GCM_SHA384
TLS Version: TLSv1.3

Domain DNS Records:
*****
[DNS] A:
98.94.89.209
[DNS] AAAA:
[DNS] MX:
[DNS] TXT:
[DNS] NS:
[DNS] CNAME:
*****

Domain DNS Servers:
*****

[+] Scanning: https://www.techie-world.net
[DNS] DIRECT OK | https://www.techie-world.net/
[TECH] Server : Apache/2.4.18 (Ubuntu)
[+] Scanning: https://www.techie-world.net/login/
[DNS] REDIRECT OK | /admin -> https://www.techie-world.net/login/
[TECH] Server : Apache/2.4.18 (Ubuntu)
[+] Scanning: https://www.techie-world.net/robots.txt
[DNS] DIRECT OK | https://www.techie-world.net/robots.txt
[TECH] Server : Apache/2.4.18 (Ubuntu)
```

Figure 2 – Sub-domain enumeration script: Information gathered – TLS certificate data, main domain IP, defending technology information, available links such as <https://www.techie-world.net/robots.txt>. Figure 3 – Sub-domain enumeration script: Inserting the target domain name for testing.



```
User-agent: *
Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:68.0) Gecko/20100101 Firefox/68.0
```

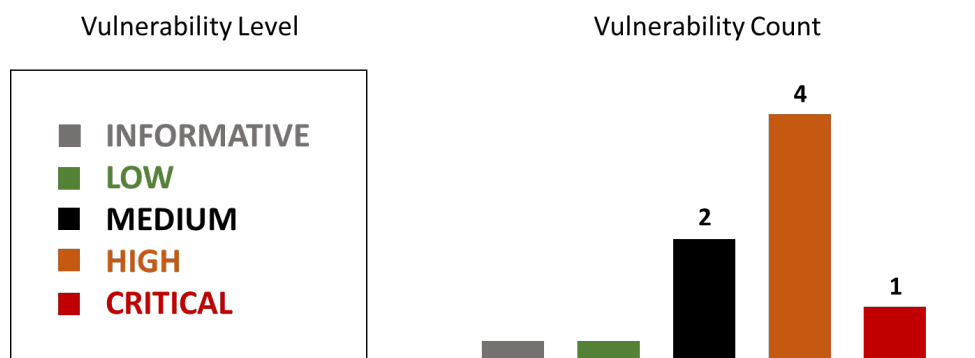
Figure 3 – Sub-domain enumeration script: Pressed on one of the links – <https://www.techie-world.net/robots.txt>.

CONCLUSIONS

From our professional view, the security level exists in the client's systems is now on average **HIGH**. Calculated number of vulnerabilities found is 7 VULN(s).

The was rated as mentioned before due to the existence of multiple vulnerabilities such as RCE **CRITICAL**, LFI **HIGH**, Sensitive Data Exposure **HIGH**.

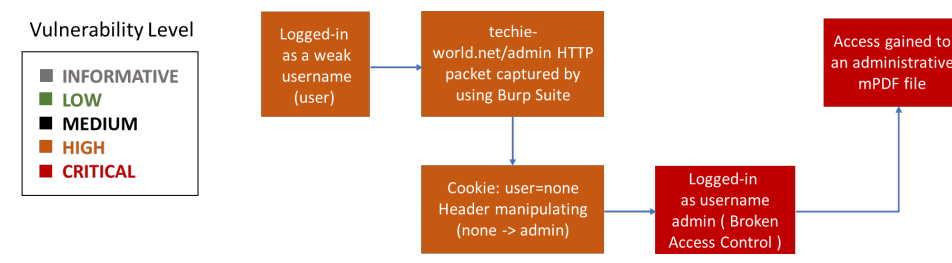
Exploiting most of the vulnerabilities mentioned above requires a mix technical knowledge.



ATTACK TREE FOR COMPLEX SCENARIOS

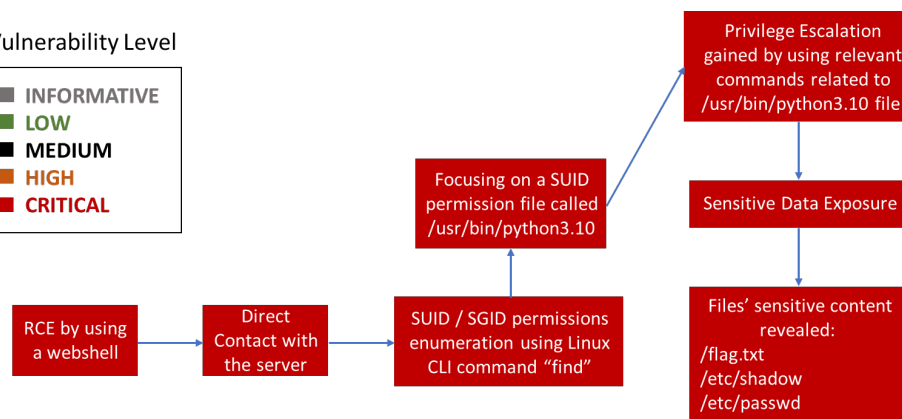
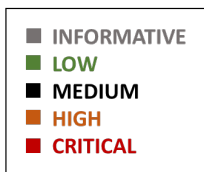
The following diagram describes each complex attack scenarios that can be applied in the client's system.

ATTACK SCENARIO NO.1 - Broken Access Control via weak protection on session cookies



ATTACK SCENARIO NO.2 - Sensitive Data Exposure via unnecessary SUID / SGID Permissions

Vulnerability Level



SETTING GOALS AND OBJECTIVES

The following paramount objectives were defined for the penetration testing engagement:

- **Data Access:** Access and retrieve the contents of the /flag.txt file.
- **Privilege Escalation:** Escalate privileges to the highest level to achieve full system takeover.

- Search for *low hanging fruits* – (**ACHIEVED**)
- Finding a *number of vulnerabilities* that could endanger the target – (**ACHIEVED**)
- Performs a vulnerability combination *perform a complex attack* to maximize the attacker's abilities - (**ACHIEVED**)
- Exposing the target to the ability to *run code remotely* - (**ACHIEVED**)
- Obtaining *Domain Admin privileges* in the target's environment - (**ACHIEVED**)
- Performing *Lateral Movement*, exposing and exploiting additional positions - (**NOT ACHIEVED**)
- *Utilization of protocols* in favor of the attacker - (**NOT ACHIEVED**)

IDENTIFIED VULNERABILITIES

VULN-001 Reflected XSS (**MEDIUM**)

Reflected XSS (Cross-Site Scripting) is a web security vulnerability. It happens when an application receives data in an HTTP request and includes that data within the immediate response in an unsafe way.

VULN-002 Information Disclosure (**MEDIUM**)

Information Disclosure (also known as Info Leakage) is a security vulnerability where a website or system accidentally reveals sensitive data to unauthorized users. This data can help an attacker understand the infrastructure and launch bigger attacks.

VULN-003 Brute-Force (**HIGH**)

Brute Force is a cyberattack method where an attacker repeatedly tries many possible passwords or combinations until the correct one is found.

VULN-004 Broken Access Control (**HIGH**)

Broken Access Control is a security vulnerability where users can access resources or perform actions they are not authorized to access.

VULN-005 Local File Inclusion (**HIGH**)

LFI (Local File Inclusion) is a vulnerability that allows an attacker to access or include files from the server by manipulating a file path. This attack can lead to RCE (Remote Code Execution).

VULN-006 Remote Code Execution (**CRITICAL**)

RCE (Remote Code Execution) is a vulnerability that allows an attacker to execute arbitrary commands or code on a remote server. This type of attack can cause a serious damage on the target machine.

VULN-007 Sensitive Data Exposure (HIGH)

Sensitive Data Exposure is a vulnerability that occurs when an application or system fails to adequately protect sensitive or confidential information at rest, in transit, or during processing. This security flaw allows unauthorized individuals to access, intercept, modify, or exfiltrate critical data.

FINDING DETAILS

VULN-001 Reflected XSS (Medium)

CVSS

Result - CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General <Medium>	Probability <High>	Severity <High>	Fix Effort <Low>
---------------------------	-----------------------------	--------------------------	---------------------------

DESCRIPTION

Cross-site scripting (XSS) is a type of security vulnerability typically found in web applications. XSS attacks enable attackers to inject client-side scripts (JavaScript and HTML content) into web pages viewed by other users. A cross-site scripting vulnerability may be used by attackers to bypass access controls such as the same-origin policy. XSS effects vary in range from petty nuisance to significant security risk, depending on the sensitivity of the data handled by the vulnerable site and the nature of any security mitigation implemented by the site's owner network.

The XSS attack vector can be divided into several different methodologies, such as:

- Reflected XSS – Occurs when a server reflecting a parameter sent from the client into the content of the page itself, without properly sanitizing the content.
- Stored XSS – Occurs when the data provided by the attacker is saved by the server, and then permanently displayed on "normal" pages returned to other users in the course of regular browsing, without proper HTML escaping.
- DOM-based XSS - As the JavaScript code was also processing user input and rendering it in the web page content, a new sub-class of reflected XSS attacks started to appear that was called DOM-based cross-site scripting. In a DOM-

based XSS attack, the malicious data does not touch the web server. Rather, it is being reflected by the JavaScript code, fully on the client side.

PROOF OF CONCEPT

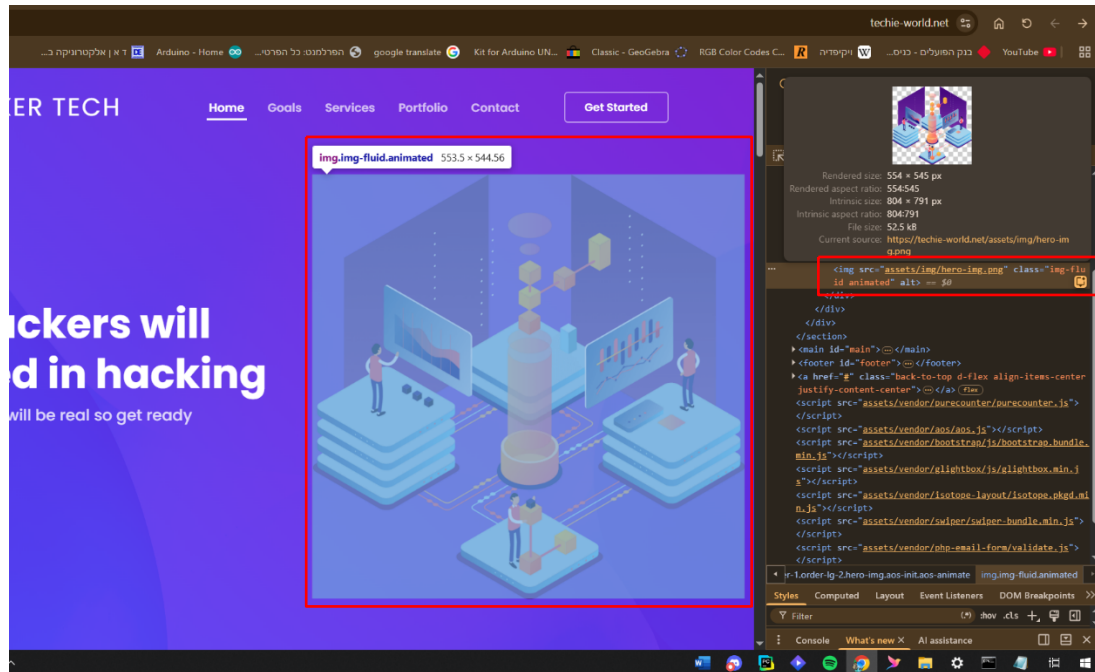


Figure 1 – Vulnerable HTML `<img>` tag to exploit.

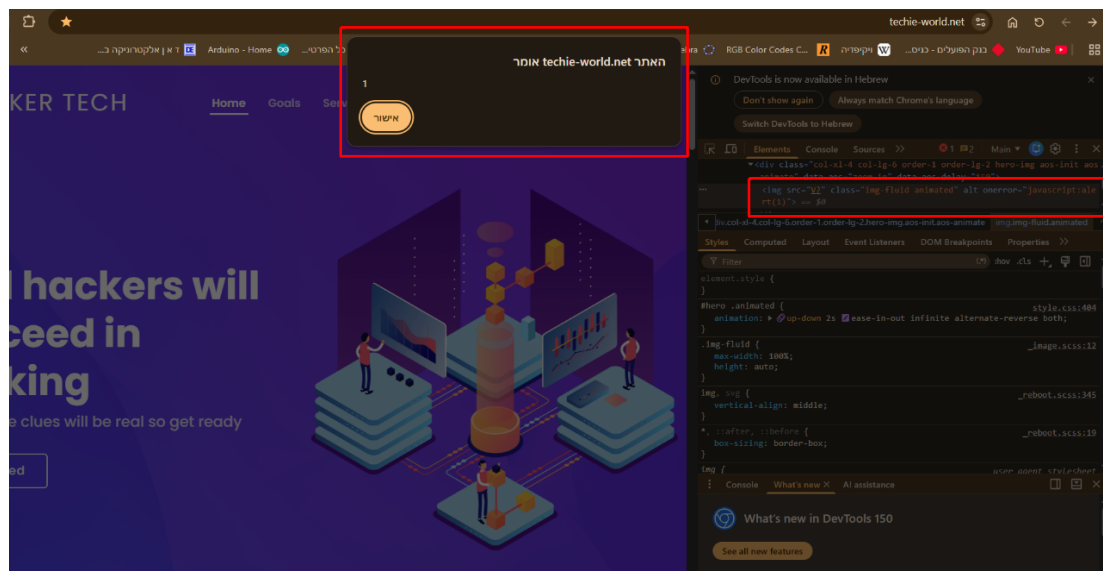


Figure 2 - `<img>` tag manipulation + alert triggered by the back-end.

DETAILS

On the website's homepage <https://techie-world.net/>, we identified a vulnerable tag that was susceptible to malicious code injection. For proof-of-concept (PoC) purposes only, we used JavaScript to inject an `alert()` command. This successfully triggered a browser alert displaying the number 1, confirming the execution of the injected code within the tag.

RECOMMENDED MITIGATIONS

- (1) Context-Aware Output Encoding** - This is the most effective defense against XSS. Before displaying any user-supplied data back to the browser, you must encode it based on where it appears in the HTML document.
- (2) Robust Input Validation and Sanitization** - Ensure the application only accepts properly formatted data before processing it.
- (3) Implement Content Security Policy (CSP)** - CSP is a powerful defense-in-depth mechanism implemented via HTTP response headers.

VULN-002 Information Disclosure (**Medium**)

CVSS

Result - CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General <Medium>	Probability <High>	Severity <Medium>	Fix Effort <Low>
--	--	---	--

DESCRIPTION

Information Disclosure (also known as Information Leakage) occurs when a website or application accidentally reveals sensitive data to unauthorized users.

This leaked information can be highly valuable to cybercriminals, who often use it as a stepping stone to launch more targeted and sophisticated attacks.

Here is a breakdown of how it happens, what data is usually leaked, and how it impacts security:

- **Technical Details:** Error messages that reveal database types, software versions, internal IP addresses, or stack traces.
- **Sensitive Files:** Publicly accessible configuration files (.env, .config, .json, etc.), source code repositories (.git), or backup files.
- **User Data:** Unintentional exposure of users' personal identifiable information (PII), such as full names, email addresses, credit card digits, or session tokens.
- **Business Logic Data:** Revealing hidden URLs, administrative paths, or internal developer comments left in the HTML source code.

PROOF OF CONCEPT

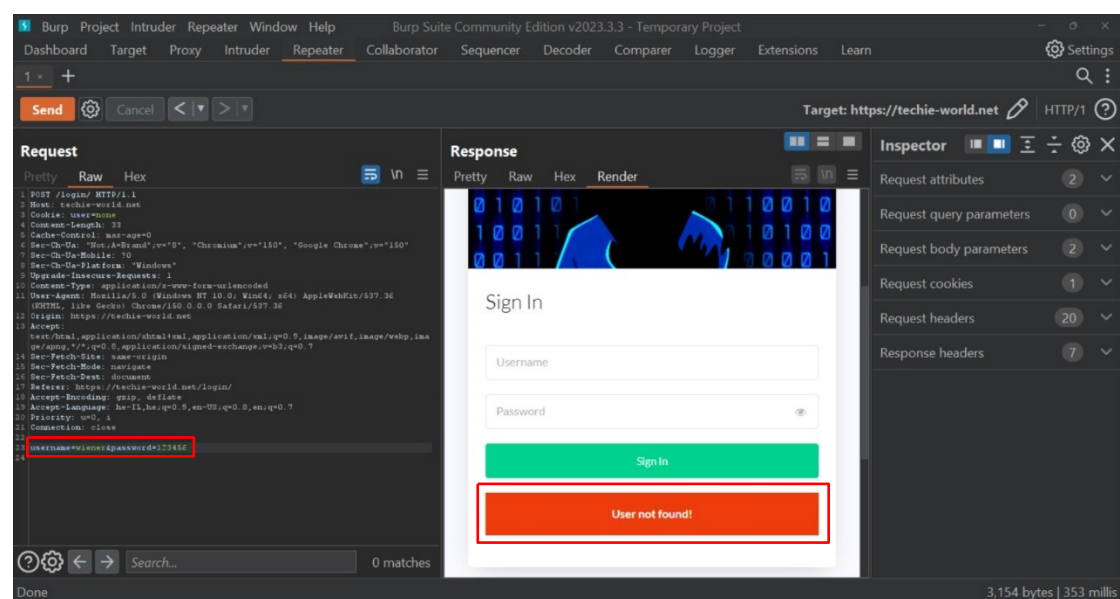


Figure 1 – Username input triggering useful message for attackers to perform user enumeration.

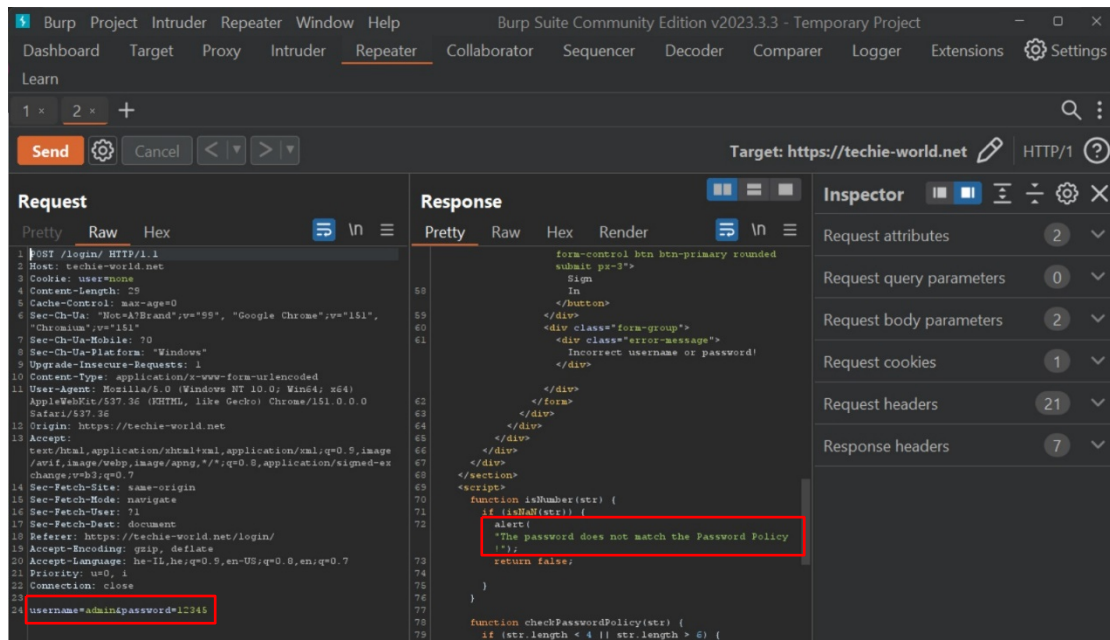


Figure 2 - Username input triggering useful message for attackers to perform password enumeration after verifying admin user is in the system.

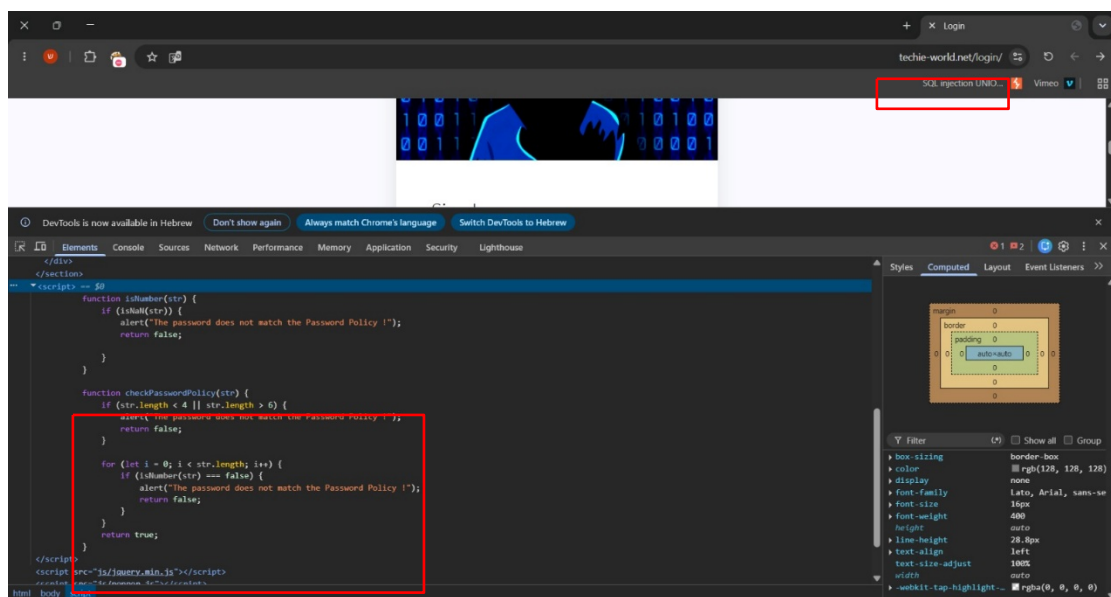


Figure 3 – Password policy script is visible to any user inspecting the source HTML which can be useful for attackers to perform password enumeration – The policy says that there must be between 4-6 characters and contain only integers.

DETAILS

During security testing of the login page <https://techie-world.net/login>, multiple username and password combinations were submitted in order to analyze the application's authentication behavior and identify potential differences in server responses.

The application returned distinguishable responses depending on the username provided. Based on these response variations, it was possible to determine which

usernames were valid and authorized within the system. This behavior enables user enumeration, allowing an attacker to identify legitimate internal accounts.

During further testing, the application's HTML content was inspected at the following path: <https://techie-world.net/login>. Sensitive information regarding the application's password policy was discovered, revealing the expected password structure and requirements.

The combination of valid username enumeration and exposed password-policy information may significantly reduce the effort required to target internal user accounts. An attacker could leverage this information to perform more focused credential-based attacks, potentially resulting in unauthorized account access, exposure of personal or sensitive information, and compromise of user privacy.

Disclosure of valid internal usernames and password requirements provides attackers with valuable information that can facilitate targeted attacks against legitimate accounts.

RECOMMENDED MITIGATIONS

- (1) **Generic Error Messages** - Use the same error message for every failed login attempt to prevent attackers from identifying valid usernames.
For example: "Username or Password are incorrect!", "Login failed", etc.
- (2) **Limit Sensitive Information Exposure** - Remove unnecessary password-policy or authentication-related information from publicly accessible HTML/source code.
- (3) **Rate Limiting** - Limit repeated login attempts to reduce the risk of automated attacks against valid user accounts.

VULN-003 Brute-Force (HIGH)

CVSS

Result - CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:L/VI:N/VA:H/SC:L/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General <High>	Probability <Medium>	Severity <High>	Fix Effort <Medium>
-------------------------	-----------------------------	--------------------------	----------------------------

DESCRIPTION

Brute Force is a cyberattack method where an attacker repeatedly tries many possible passwords or combinations until the correct credentials are found.

This attack is particularly effective when an authentication mechanism lacks protections. Successful brute-force attack may result in unauthorized access to user accounts and sensitive information.

PROOF OF CONCEPT

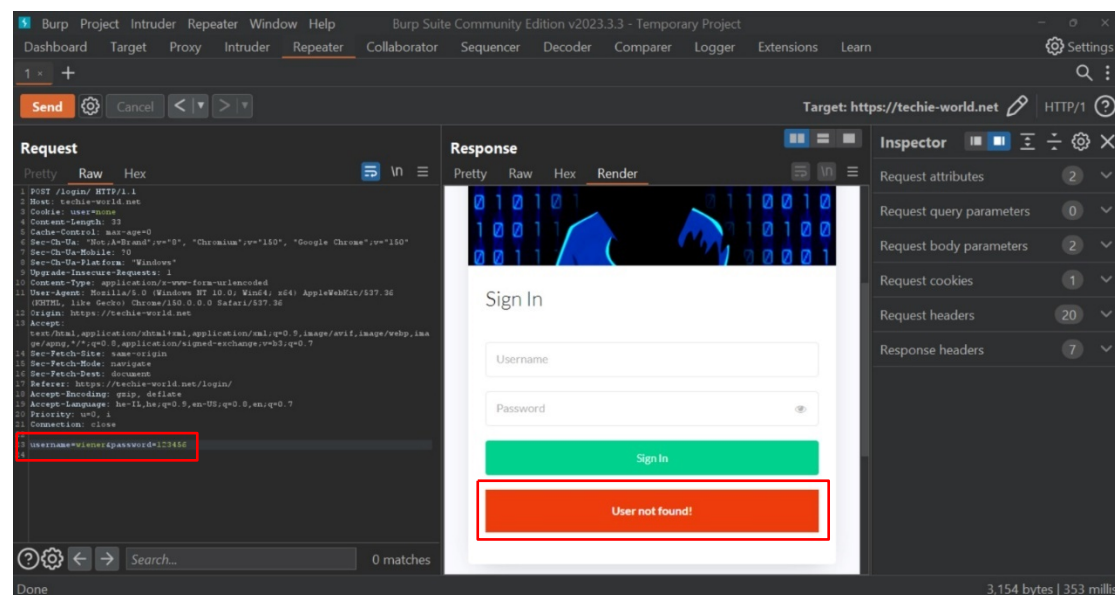


Figure 1 – Username input triggering useful message for attackers to perform user enumeration.

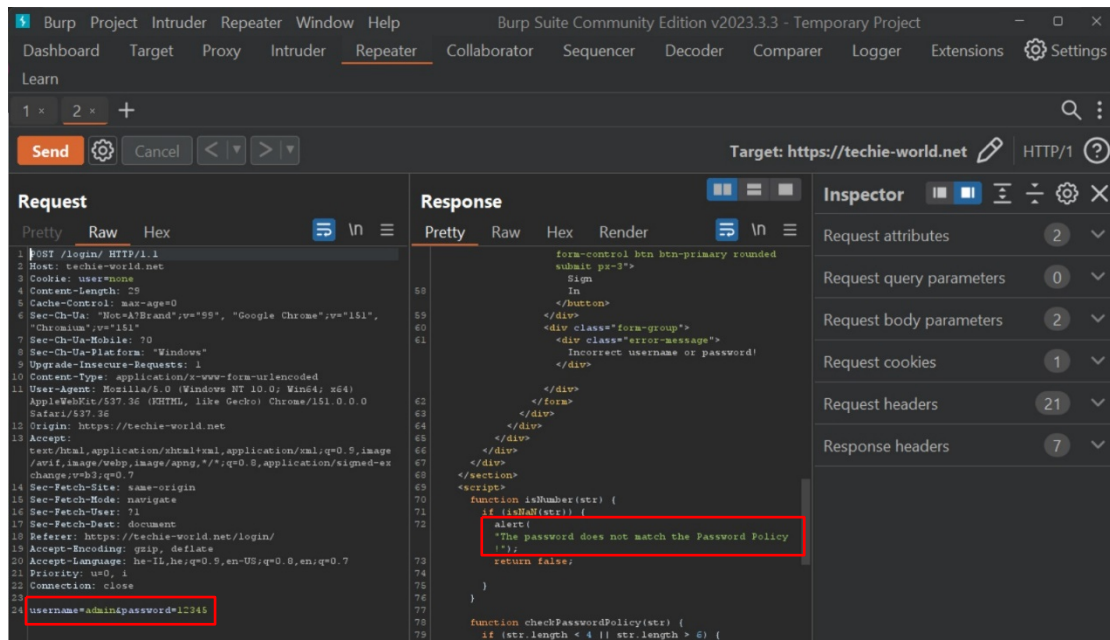


Figure 2 - Username input triggering useful message for attackers to perform password enumeration after verifying admin user is in the system.

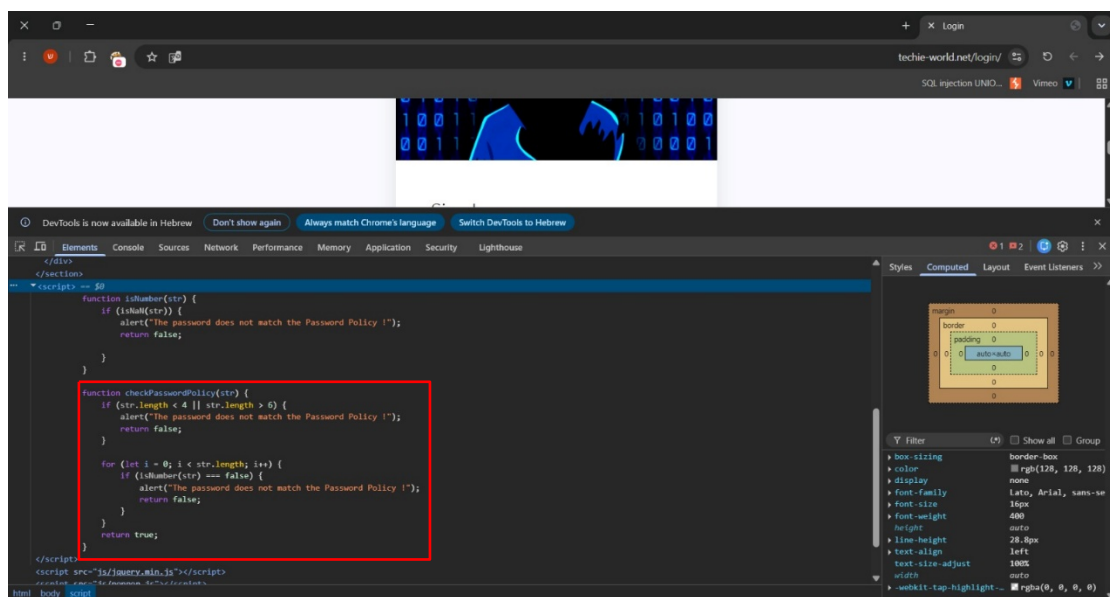


Figure 3 – Password policy script is visible to any user inspecting the source HTML which can be useful message for attackers to perform password enumeration – The policy says that there must be between 4-6 characters and contain only integers.

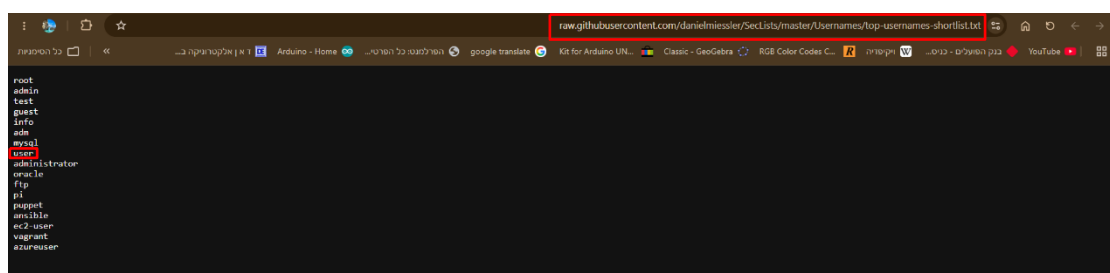


Figure 4 – Username list for enumeration found in GitHub.com

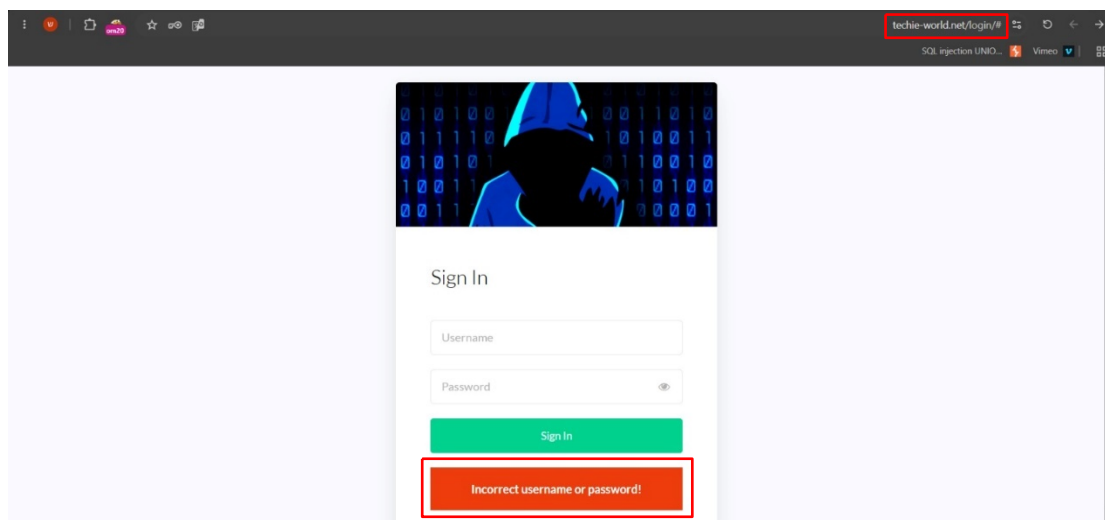


Figure 5 – Username input triggers a suspicious message about logging-in as Username=user - which we can start a password enumeration with the relevant username.



2. Intruder attack of https://techie-world.net - Temporary attack - Not saved to project file							
Results Positions Payloads Resource pool Settings							
Filter: Showing all items							
Request	Payload	Status	Error	Timeout	Length	Comment	
0		200	☐	☐	3170		
1	1	200	☐	☐	3170		
2	2	200	☐	☐	3170		
3	3	200	☐	☐	3170		
4	4	200	☐	☐	3170		
5	5	200	☐	☐	3170		
6	6	200	☐	☐	3170		
7	7	200	☐	☐	3170		
8	8	200	☐	☐	3170		
9	9	200	☐	☐	3170		
10	10	200	☐	☐	3170		
11	11	200	☐	☐	3170		
12	12	200	☐	☐	3170		
13	13	200	☐	☐	3170		
14	14	200	☐	☐	3170		
15	15	200	☐	☐	3170		
16	16	200	☐	☐	3170		
17	17	200	☐	☐	3170		

Figure 6 – Burp Suite Intruder sending huge amount of requests (263 packets) to /techie-world.net domain at the /login path – which could indicate a brute-force attack.

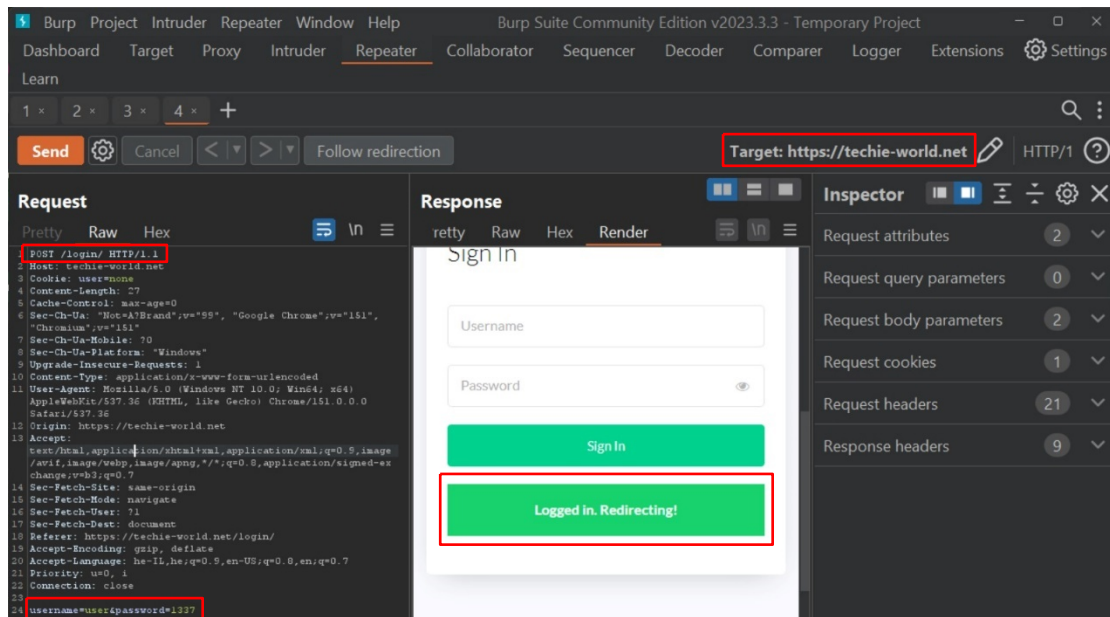


Figure 7 – Brute-Force attack completed, username + password credentials are found – which redirecting to user's account page.

DETAILS

Initially, we discovered a list of usernames at the following path:

<https://raw.githubusercontent.com/danielmiessler/SecLists/master/Usernames/top-usernames-shortlist.txt>, which was identified through GitHub.com. This list was used to perform username enumeration against the application's login functionality at techie-world.net/login.

During the enumeration process by using Burp Suite Intruder, attempts with invalid usernames returned the error message "Username not found!". However, when testing the usernames "user" and "admin", the application returned a different response: "Incorrect username or password!". This difference in authentication responses indicated that these usernames were likely valid accounts within the system, allowing us to focus further password testing on them.

Next, we inspected the HTML source code of the /login page and identified a client-side script that disclosed the application's password requirements (password policy). Based on the exposed validation logic, we determined that passwords were required to contain 4–6 numeric characters only.

Using this information, we configured Burp Suite Intruder to perform a brute-force attack against the identified valid accounts, testing numeric password combinations within the disclosed password policy. The attack was successful, and valid credentials were identified: Username=user + Password=1337, which allowed us to successfully authenticate and gain access to user's account page.

RECOMMENDED MITIGATIONS

- (1) **Implement Rate Limiting and Account Lockout** - Limit the number of failed login attempts and temporarily lock or delay additional attempts after multiple authentication failures to mitigate brute-force attacks.
- (2) **Use Generic Authentication Error Messages** - Return the same error message, such as "Invalid username or password", regardless of whether the username exists. This prevents attackers from performing username enumeration.
- (3) **Enforce a Strong Password Policy** - Require longer and more complex passwords instead of short numeric-only passwords, significantly increasing the number of possible password combinations.
- (4) **Avoid Exposing Sensitive Authentication Logic** - Password policies and validation mechanisms should not rely solely on client-side code. Sensitive validation and authentication controls should be enforced securely on the server side only.
- (5) **Implement Multi-Factor Authentication (MFA)** - Require an additional authentication factor, especially for sensitive or privileged accounts, reducing the impact of compromised credentials.

VULN-004 Broken Access Control (HIGH)

CVSS

Result - CVSS:4.0/AV:N/AC:L/AT:N/PR:L/UI:N/VC:H/VI:L/VA:L/SC:H/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General <High>	Probability <Low>	Severity <High>	Fix Effort <Low>
-------------------------	----------------------------	--------------------------	---------------------------

DESCRIPTION

Broken Access Control is a security vulnerability where users can access resources or perform actions they are not authorized to access.

This weakness may allow an attacker to access unauthorized resources, view or modify sensitive information, perform restricted actions, or gain higher privileges than intended. Broken Access Control can result from improper permission checks, predictable resource identifiers, exposed administrative functionality, or insufficient authorization validation on the server side.

PROOF OF CONCEPT

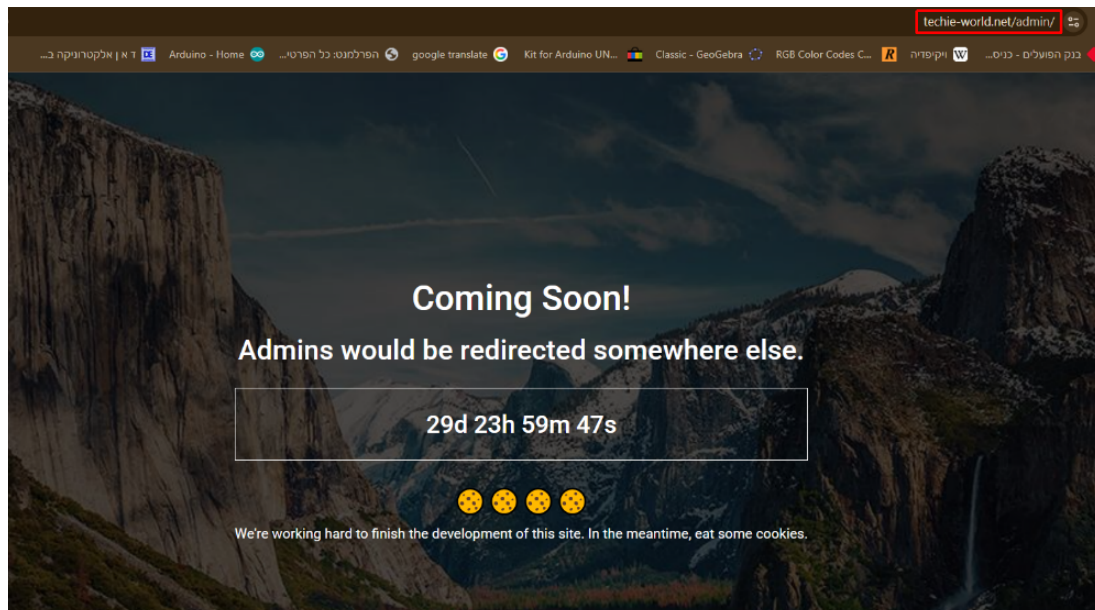


Figure 1 – Start point: techie-world.net/admin (Username=user account page).

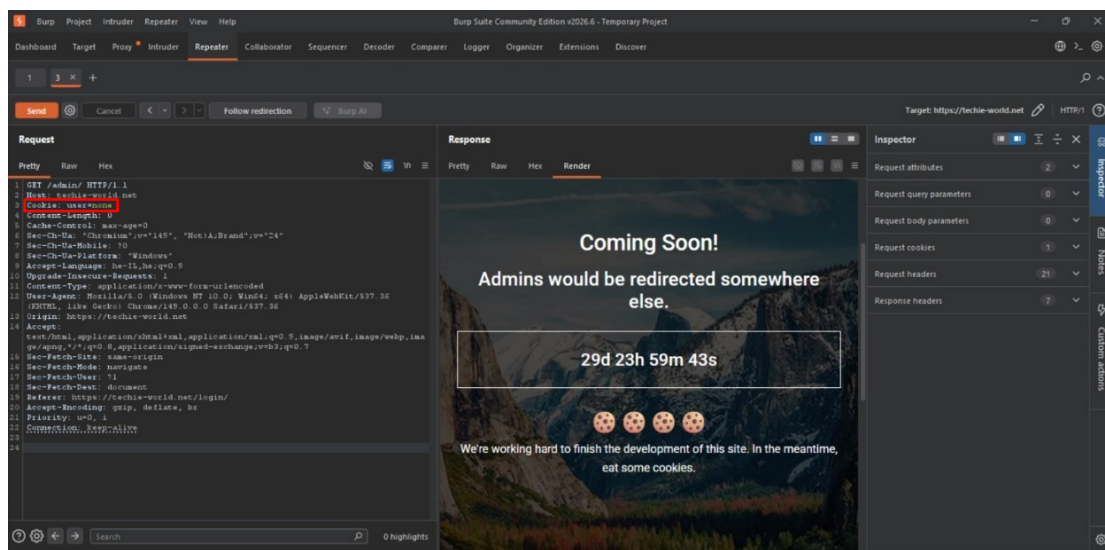


Figure 2 – HTTP original packet after logging-in as user.

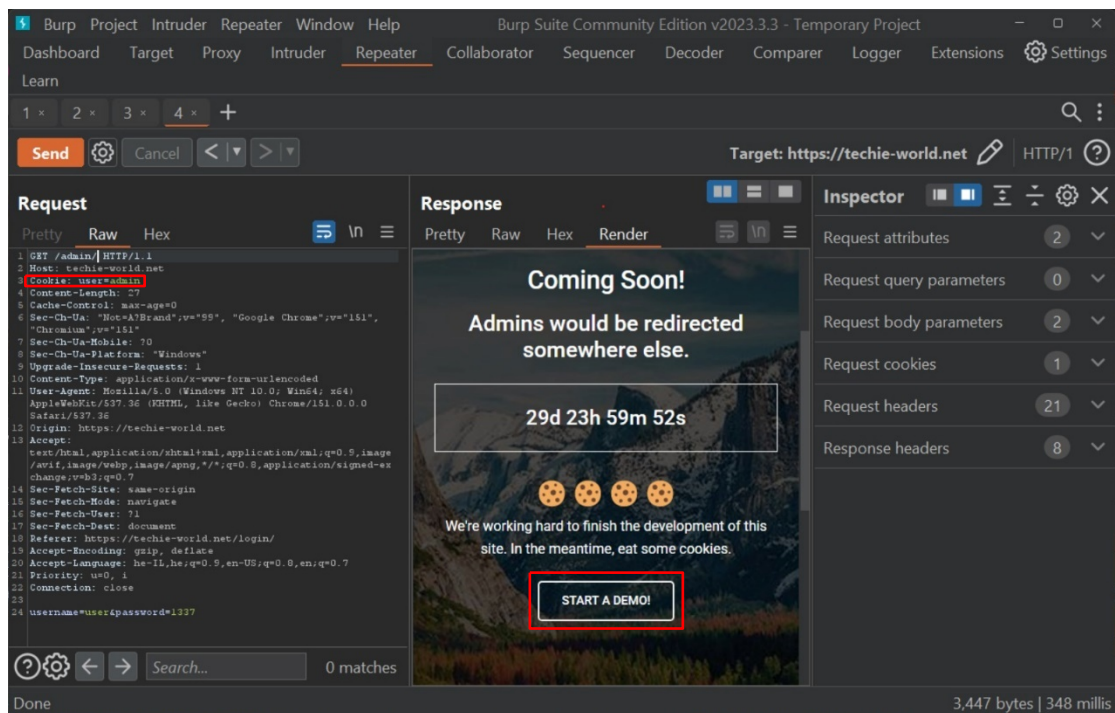


Figure 3 – HTTP packet after manipulating the Cookie: user=none Header to user=admin – which gained us access to admin's account page.

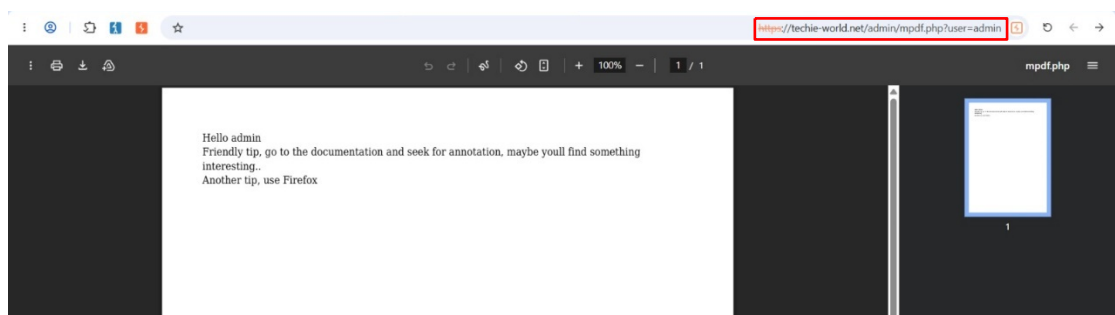


Figure 4 – After pressing on: "START A DEMO!" – access gained to a PDF administrative file.

DETAILS

The attack was initiated from the low-privileged user's account page at: techie-world.net/admin. Using Burp Suite Proxy, the outbound HTTP request targeting this authenticated path was intercepted.

Inspection of the request headers revealed a session cookie parameter formatted as `Cookie: user=none`. To evaluate the integrity of the session validation mechanism, parameter tampering was conducted by modifying the cookie value from none to admin.

Upon forwarding the manipulated request, the application accepted the tampered header without proper server-side verification. This allowed a complete authentication bypass, resulting in unauthorized access to the administrator

interface and full account takeover. At this point we gained access to a PDF file in administrative panel.

RECOMMENDED MITIGATIONS

- (1) Implement Cryptographically Secure Session IDs** - Cease storing identifiable user information (e.g., usernames or roles) directly within cookies. Instead, issue cryptographically strong, pseudo-random session tokens generated by a secure server-side session manager.
- (2) Server-Side Authorization Checks** - Ensure that access control decisions and user identity verification rely strictly on server-side session state rather than client-controlled values.
- (3) Cryptographic Signing (if using Stateless Tokens / JWTs)** - If state information must be stored client-side, utilize cryptographically signed tokens (e.g., signed JWTs with strong algorithms) and rigorously validate the signature on every request.

VULN-005 Local File Inclusion (HIGH)

CVSS

Result - CVSS:4.0/AV:N/AC:H/AT:N/PR:H/UI:P/VC:H/VI:H/VA:L/SC:H/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General <High>	Probability <Medium>	Severity <Critical>	Fix Effort <Low>
-------------------------	-----------------------------	------------------------------	---------------------------

DESCRIPTION

Local File Inclusion (LFI) is a server-side security vulnerability that occurs when an application receives, reads, or includes local system files based on improperly sanitized user-supplied input by manipulating a file path.

Exploiting an LFI vulnerability allows an attacker to navigate outside the intended directory structure and access arbitrary, sensitive files stored on the underlying server—such as application source code, configuration files containing credentials, system environment variables, and cryptographic keys. In severe cases, LFI can be leveraged to achieve Remote Code Execution (RCE).

PROOF OF CONCEPT

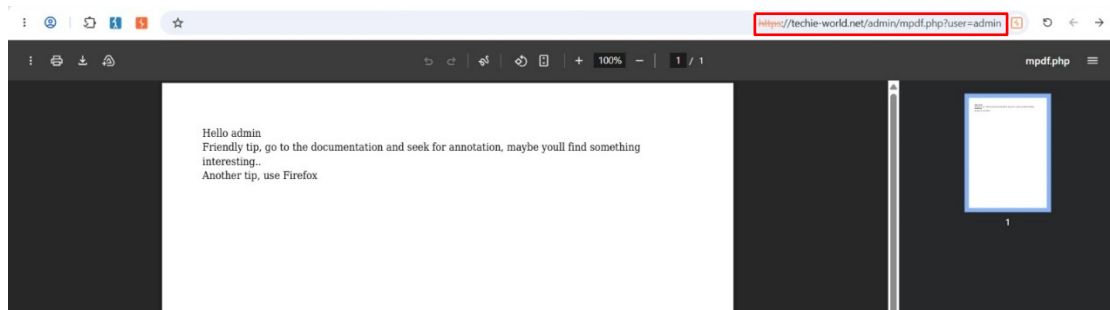


Figure 1 – Start point: Access gained to a PDF admin file from the administrative panel.

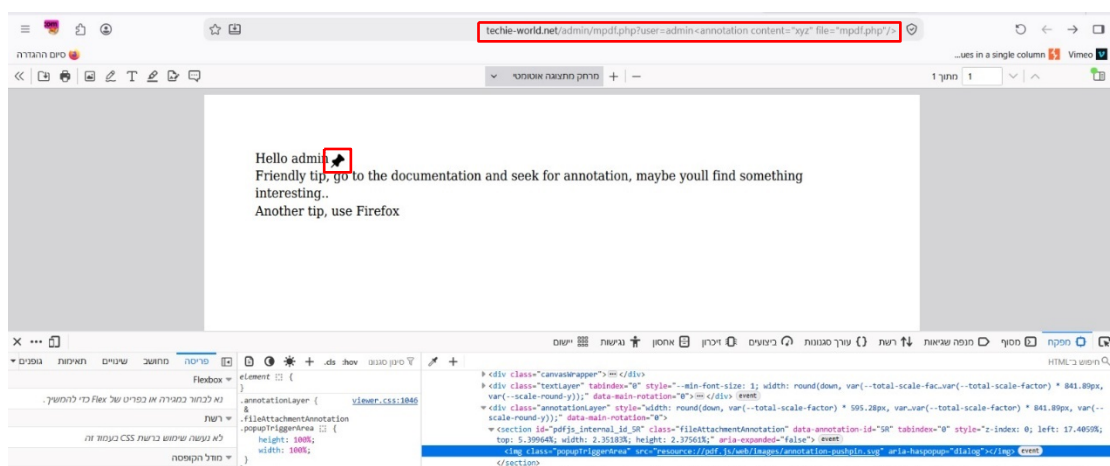


Figure 2 – Manipulating the <annotation> tag in the URL to trigger the Annotation note functionality (the pin icon).

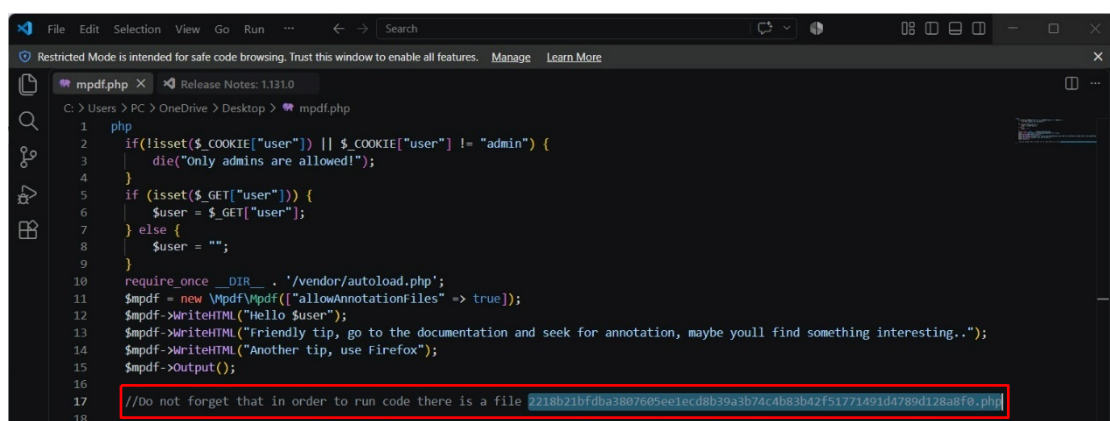


Figure 3 – Double click on the Annotation note functionality (the pin icon), downloads this .php file which was sent from the back-end. The marked comment says that there is a sensitive file in the same path at the back-end.

DETAILS

During the assessment, upon gaining access to a PDF page within the admin panel, we attempted to inject an <annotation> tag via a URL parameter. This successfully rendered an embedded pin icon with functional link capabilities on the document.

Double clicking the pin icon exposed an option to download a sensitive backend PHP file directly to the testing computer. In a broader exploitation scenario, this capability could allow an attacker to retrieve critical system files, potentially leading to severe system impact or complete full-system compromise.

In this instance, inspection of the downloaded file revealed a code comment referencing an existing PHP file. Leveraging this disclosure ultimately enabled the invocation and execution of a web shell on the target server.

RECOMMENDED MITIGATIONS

- (1) Input Validation and Allowlisting** - Validate all user-supplied input and allow only predefined and trusted file names or paths. Path traversal sequences such as ../ and unauthorized absolute paths should be rejected.
- (2) Avoid Direct User Input in File Operations** - User-controlled input should never be passed directly to functions responsible for loading, including, or reading files. Instead, use predefined mappings between allowed input values and server-side files.
- (3) Restrict File System Permissions** - Apply the Principle of Least Privilege to the web application and web server. The application should only have access to files required for normal operation, reducing the potential impact of a successful LFI attack.

VULN-006 Remote Code Execution (**CRITICAL**)

CVSS

Result - CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:H/SI:H/SA:H

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General | <Critical> Probability <High> Severity | <Medium> Fix Effort | <Low>

DESCRIPTION

RCE (Remote Code Execution) is a vulnerability that allows an attacker to execute arbitrary commands or code on a remote server. This type of attack can cause a serious damage on the target machine.

RCE is one of the most critical vulnerabilities in cybersecurity. It occurs when an attacker is able to execute arbitrary commands or code (e.g., Bash, PHP, Python, PowerShell, etc.) on a remote server or target machine over a network, often without requiring prior authentication or physical access.

This type of cyber-attack can cause real damage on the target machines it interacts with and even leak sensitive data.

PROOF OF CONCEPT

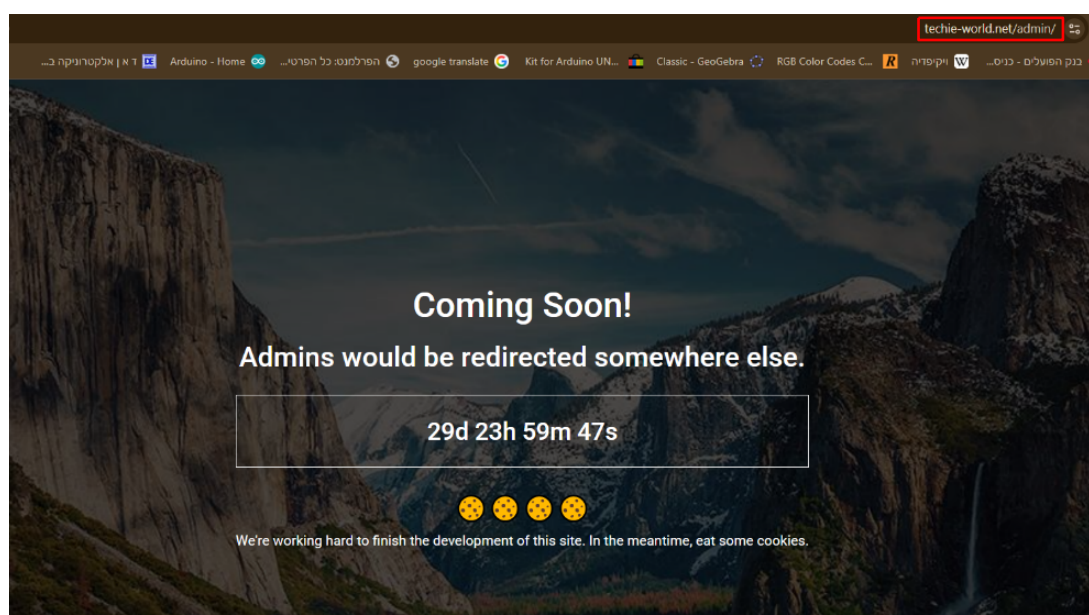


Figure 1 – Start point: techie-world.net/admin (Username=user account page).

```

1  php
2  if(isset($_COOKIE["user"]) || $_COOKIE["user"] != "admin") {
3      die("Only admins are allowed!");
4  }
5  if (isset($_GET["user"])) {
6      $user = $_GET["user"];
7  } else {
8      $user = "";
9  }
10 require_once __DIR__ . '/vendor/autoload.php';
11 $mpdf = new \Mpdf\Mpdf(["allowAnnotationFiles" => true]);
12 $mpdf->WriteHTML("Hello $user");
13 $mpdf->WriteHTML("Friendly tip, go to the documentation and seek for annotation, maybe you'll find something interesting..");
14 $mpdf->WriteHTML("Another tip, use Firefox");
15 $mpdf->Output();
16
17 //Do not forget that in order to run code there is a file 2218b21bfd8a3807605ee1ecd8b39a3b74c4b83b42f51771491d4789d128a8f0.php
18

```

Figure 2 – Double click on the Annotation note functionality (the pin icon), downloads this .php file which was sent from the back-end. The marked comment says that there is a sensitive file in the same path at the back-end (using the previous LFI attack methodology).

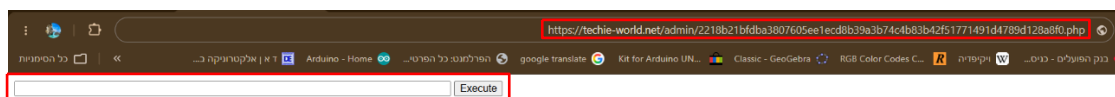


Figure 3 – By inserting the file name existing in the back-end (.php), we successfully spawned a web shell – which can interact with the entire file system of the internal server.

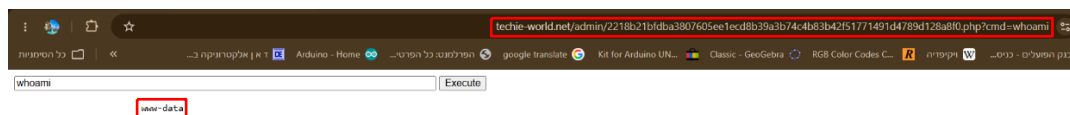


Figure 4 – Linux CLI command: Executing the command "whoami" to see what user we logged-in with.

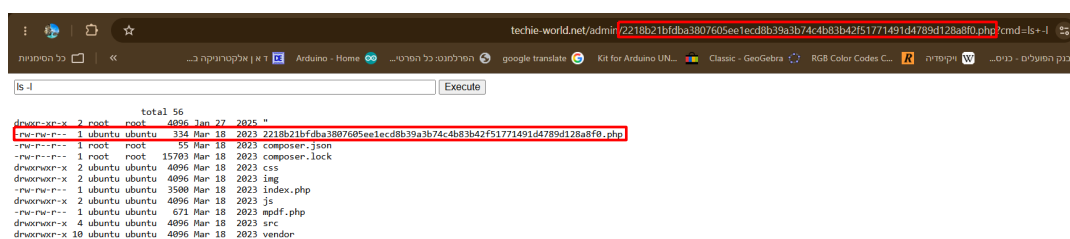


Figure 5 – Linux CLI command: Executing the command "ls -l" to see what files and directories are in the current directory. Including the marked web shell (.php) file we took over.

DETAILS

Following the execution of the LFI (Local File Inclusion) attack detailed earlier in this report, analysis of the backend file retrieved to the testing machine revealed a code comment referencing a specific PHP file existing in the server (see Figure 2).

We subsequently identified an exposed endpoint under the target domain: techie-world.net/ that was vulnerable via URL parameter manipulation. This allowed us to inject the path of the secondary PHP file disclosed in the downloaded file, ultimately resulting in successful Web Shell execution by the same file name.

For Proof of Concept (PoC) purposes, several sample system commands were executed, including "whoami" and "ls -l" (see Figures 4, 5).

Had a malicious actor (Black Hat) exploited this vulnerability, it could have enabled arbitrary system command execution, leading to complete domain compromise and total system disruption.

RECOMMENDED MITIGATIONS

- (1) Input Validation & Sanitization** - Implement strict allow-listing (whitelisting) for all user inputs.
- (2) Avoid Dangerous Functions** - Do not pass user input directly to system execution functions. For example: eval(), exec(), system(), etc.
- (3) Principle of Least Privilege** - Run web applications under restricted system user accounts without OS shell access or write permissions to source code directories.

VULN-007 Sensitive Data Exposure (HIGH)

CVSS

Result - CVSS:4.0/AV:L/AC:L/AT:N/PR:L/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

Calculator - <https://www.first.org/cvss/calculator/4.0>

RISK

General | <High>

Probability <Medium>

Severity | <Critical>

Fix Effort | <Low>

DESCRIPTION

Sensitive Data Exposure is a vulnerability that occurs when an application or system fails to adequately protect sensitive or confidential information at rest, in transit, or during processing. This security flaw allows unauthorized individuals to access, intercept, modify, or exfiltrate critical data.

PROOF OF CONCEPT

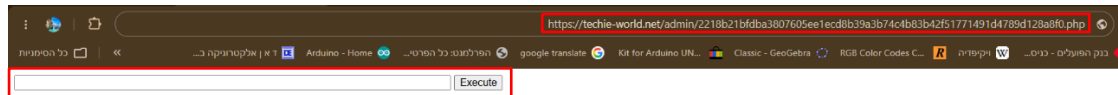


Figure 1 – Start point: Spawned Web Shell command line – Ubuntu/Linux OS internal server interaction.

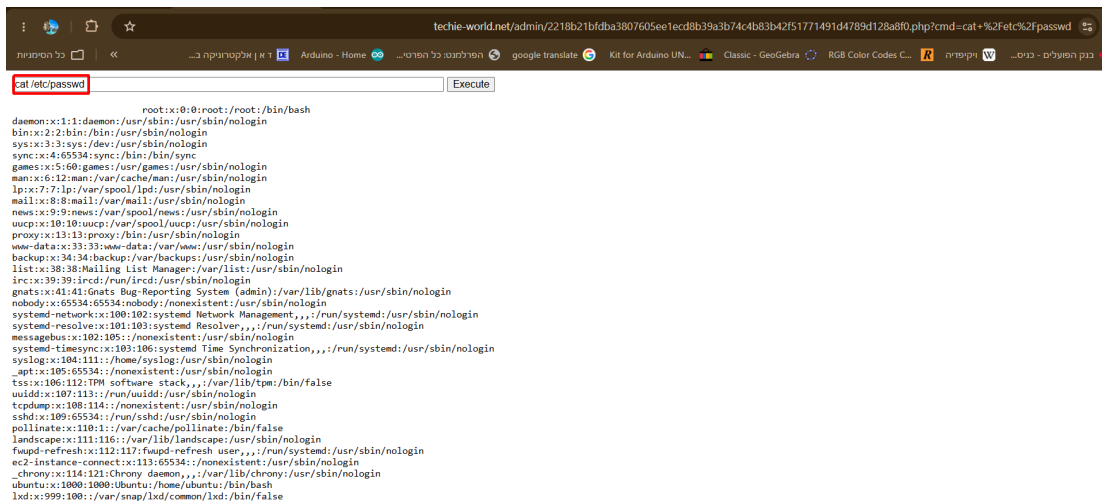


Figure 2 – Linux CLI command: Executing the command "cat" to see the data of the sensitive file /etc/passwd.

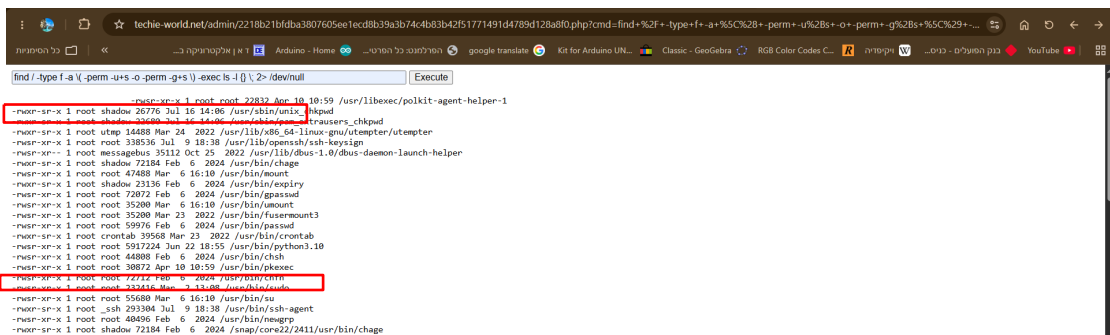
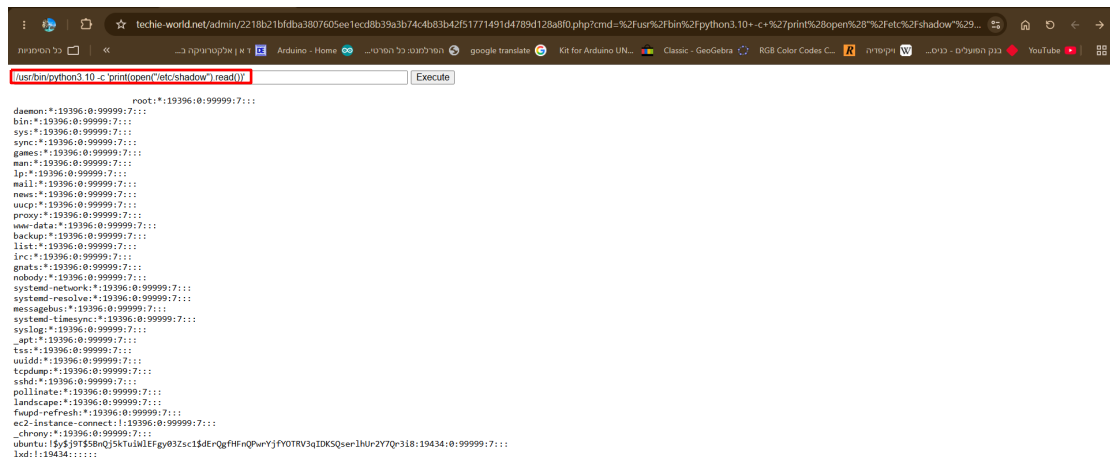
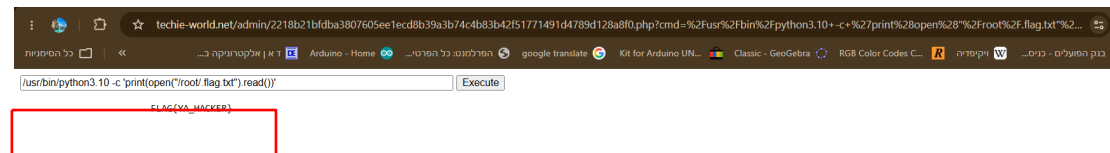


Figure 3 – Linux CLI command: Executing the command "find" with the relevant flags to see all files over the system with SUID / SGID permissions – After enumerating, we focused on a file name "/usr/bin/python3.10".



The screenshot shows a web browser window with the URL `techie-world.net/admin/2218b21bfdba380760see1ecd8b39a3b74c4b83b42f51771491d4789d128a8f0.php?cmd=%2Fusr%2Fbin%2Fpython3.10+-c+%27print%28open%28"%2Fetc%2Fshadow"%29...`. The terminal window displays the command `/usr/bin/python3.10 -c 'print(open("/etc/shadow").read())'` and its output, which is a list of system users and their hashed passwords, including `daemon:*,19396:0:99999:7:::`, `bin:*,19396:0:99999:7:::`, `sys:*,19396:0:99999:7:::`, `sync:*,19396:0:99999:7:::`, `games:*,19396:0:99999:7:::`, `man:*,19396:0:99999:7:::`, `lp:*,19396:0:99999:7:::`, `mail:*,19396:0:99999:7:::`, `news:*,19396:0:99999:7:::`, `uucp:*,19396:0:99999:7:::`, `proxy:*,19396:0:99999:7:::`, `www-data:*,19396:0:99999:7:::`, `backup:*,19396:0:99999:7:::`, `list:*,19396:0:99999:7:::`, `irc:*,19396:0:99999:7:::`, `gnats:*,19396:0:99999:7:::`, `nobody:*,19396:0:99999:7:::`, `systemd-network:*,19396:0:99999:7:::`, `systemd-resolve:*,19396:0:99999:7:::`, `messagebus:*,19396:0:99999:7:::`, `systemd-timesync:*,19396:0:99999:7:::`, `syslog:*,19396:0:99999:7:::`, `apt:*,19396:0:99999:7:::`, `tas:*,19396:0:99999:7:::`, `uid64:*,19396:0:99999:7:::`, `tcpdump:*,19396:0:99999:7:::`, `sshd:*,19396:0:99999:7:::`, `pollinate:*,19396:0:99999:7:::`, `landscape:*,19396:0:99999:7:::`, `fwupd-refresh:*,19396:0:99999:7:::`, `ec2-instance-connect:*,19396:0:99999:7:::`, `_chrony:*,19396:0:99999:7:::`, `_ubuntu:18:519158nQj5kTuiWlEFgy63Zsc1$deRqgfhFwQhuryjfyOTRv3qIDKSqer1Au-2Y7Qr318:19434:0:99999:7:::`, and `1ad:1:19434:::`.

Figure 4 – Linux CLI command: Executing a related command to `"/usr/bin/python3.10"` file called `"print()"` with the relevant flags to see the content of the sensitive file `"/etc/shadow"`.



The screenshot shows a web browser window with the URL `techie-world.net/admin/2218b21bfdba380760see1ecd8b39a3b74c4b83b42f51771491d4789d128a8f0.php?cmd=%2Fusr%2Fbin%2Fpython3.10+-c+%27print%28open%28"%2Froot%2Fflag.txt"%2...`. The terminal window displays the command `/usr/bin/python3.10 -c 'print(open("/root/flag.txt").read())'` and its output, which is `FLAG{VA_HACKER}`.

Figure 5 – Linux CLI command: Executing a related command to `"/usr/bin/python3.10"` file called `"print()"` with the relevant flags to see the content of the sensitive file `"/flag.txt"` as requested at our CTF final task at ECOM college.

DETAILS

During the test, a vulnerability in the web application was identified that allowed Remote Code Execution (RCE) on the target server. By exploiting this vulnerability, it was possible to obtain a web shell and execute operating system commands within the context of a low-privileged user (username=www-data).

During the test, username=www-data succeeded reading the content of "/etc/passwd" sensitive file by using the command "cat". Following the initial compromise, the system was enumerated to identify potential privilege escalation opportunities by using the command: "find / -type f -a \(-perm -u+s -o -perm -g+s \) -exec ls -l {} \; 2>/dev/null" to see all files over the system with SUID / SGID permissions. During this process, a SUID-enabled executable file called "usr/bin/python3.10" was identified. Due to its insecure configuration, the executable could be abused to perform Local Privilege Escalation, allowing the attacker to move from the initially compromised low-privileged user to a higher-privileged context (username=root).

After successfully escalating privileges, we gained significantly greater control over the target system and were able to access files and resources that were not accessible to the original user (www-data). This included sensitive system files such as "/etc/shadow" and "/flag.txt" as requested at our CTF final task at ECOM college, by the following commands: "usr/bin/python3.10 -c 'print(open("/etc/shadow").read())'" , "usr/bin/python3.10 -c 'print(open("/root/.flag.txt").read())'" .

RECOMMENDED MITIGATIONS

- (1) Enforce Proper Access Controls** - Restrict access to sensitive system files and backend endpoints based on the Principle of Least Privilege.
- (2) Remove Unnecessary SUID Permissions** - Remove the SUID bit from the affected executable if it is not strictly required for its intended functionality.
- (3) Audit SUID/SGID Binaries** - Review all SUID/SGID binaries across the system and remove unnecessary privileged permissions.
- (4) Enforce the Principle of Least Privilege** - Ensure that applications and users receive only the permissions necessary to perform their intended functions.

APPENDICES

METHODOLOGY

The work methodology of our penetration testing team includes some of the following potential inspected information according to the client's needs:

APPLICATIVE PENETRATION TESTS

The test was conducted identify the following:

- Vulnerable functions used in the code.
- Un-sanitized Input provided by the user.
- Well known vulnerabilities exists in the system.
- Insecure error handling.
- Unhandled manipulation that can be used by an attacker.
- Sensitive information leakage.

Performed general inspection of the code if requested by the client. In addition to the usage of automated tools to identify vulnerabilities and potential issues in the target application.

Understanding the system logic – Before performing the test, the testers watched and examined the system in order to understand its purpose and mode of operation. During this exam the examiners try to understand the following:

- **Client Requests:**
 - Examined hidden parameters.
 - Examine important parameters that are in outgoing requests
 - Notice all the request titles heading towards the server
 - Examine paths and form of loading of data on the site
- **Server Answers:**
 - Examine the number of errors that recur from the site.
 - Examine when the server returns redirection in order to find *Open Redirect*.

● **Understanding the customer side of the system:**

- The testers examined what could be done on the customer side of the system. Also, in what language is the system written and are there any comments in the client-side code.
- The testers examined which JavaScript functions are called in the code.
- Examined whether HTML code can be injected next to a client.

● **Data collection and scanning:**

- The testers examined whether there was information across the Internet about the system using various search engines.
- Usernames or employee names for Brute-Force deployment.
- Find additional servers and get information about those servers.
- Scans were also performed by dedicated tools in order to find known vulnerabilities on the site.

● **Checking the user's identity management and authorization:**

- The examiners examined the permission level in the system, what permission level they are at and whether it is possible to switch to another permission level.
- In addition, we examined whether there are different APIs that allow manipulation of the authorization level in the system or do not check the authorization level at all.

● **Checking the user authentication process:**

- The testers examined the mechanism of connection to the system, whether there is Anti-Automation protection such as CAPTCHA.

● **Authentication of the resulting input:**

- The testers examined the user's call management in addition to verifying the inputs sent from the client alongside the server. Attempts were also made and exploits of systems to upload documents to the system, file reading systems and even injecting malicious code into the system.

● **Error management in the system:**

- o During the test, errors that were repeated by a customer were identified and conclusions were drawn according to the same errors that helped the testers during this test.
- **Testing of potential attack vectors, and provideing a working POC for examination.**
- **The test result is a detailed report contains all the findings details about the vulnerabilities found:**
 - o CVSS
 - o RISK
 - o DESCRIPTION.
 - o POC
 - o DETAILS
 - o RECOMMENDED MITIGATIONS
- **Additionally, the following elements may be performed due to the client's request:**
 - o Providing the development team from "ECOM" to support the client during the mitigation process.
 - o Providing the penetration testing team from "ECOM" explain in more depth about the report.

INFRASTRUCTURE PENETRATION TEST

The test was performed in a format that would allow the company to identify the main risk points that exist in the systems and infrastructures of the company under test and treat them in a way that will allow it to reduce the chance of realizing exposure to harm and leak information into the company's and businesses.

The computer systems test was performed in five main stages:

- **First stage** – an overview of the existing computer system and mapping of all the components in the computer system and the information processing processes.

- **Second stage** – planning the test stages as a result of the mapping.
- **Third stage** – comprehensive technological tests and processes of the various components.
- **Fourth stage** – sorting and analyzing the risk outline.
- **Fifth stage** – risk assessment and corrective recommendations.
- **For the purposes of documenting the existing situation:**
 - The security survey was conducted while studying the computer system in the demarcation of the test and how it operates on the basis of conducting questioning and examining various relevant factors and operational processes. In addition, various components and technological means related to the information systems and relevant to the various survey topics were examined.
- **The questioning and documentation:**
 - was carried out in a way that enabled learning and understanding of all the existing and implemented administrative and operational processes in practice in everything related to the security of the information systems in the organization.
- **The examination of the technological issues:**
 - was carried out in a way that enabled us to become familiar with the protection circuits in the system and their practical implementation, through the use of logical and physical security measures as well as the configuration of the hardening components of the technological system.
- **Resilience tests (optional):**
 - simulate a potential intruder into the information systems, for the purpose of examining the quality of the application of the parameters and the logical security measures of the various technological components.

FINDINGS CLASSIFICATIONS

The purpose of the presentation in the manner illustrated above is on several levels:

1. **The vulnerability name** - A main vulnerability of which an examination is performed.
2. **Description of the test** - Main description about the vulnerability.
3. **Findings of the test** - Findings that clearly and concisely describe an existing situation. The purpose of the section is to document the existing situation as found during the examination. The test results can be normal or in a status that endangers the entire array tested, at the level of exposure to damage to activity continuity, leaked sensitive information or damage to property and people.
4. **The risks as a result of the existing situation** - A rating that clarifies what is the risk arising to the customer from the findings.
5. **Severity of the damage** - The method of determining the level of damage is performed according to the following details:

Critical – For the following risks:

- o The realization of the risk will lead to a horizontal impairment in the information availability of the organization's systems and / or infrastructure.
- o The realization of the risk will lead to the disclosure of information that may threaten the stability of the organization or endanger human lives.
- o Unauthorized disruption / alteration of information that may threaten the stability of the organization or endanger human life.

High - For the following risks:

- o The realization of the risk will impair the information availability of a sensitive system.
- o Exposure of sensitive information.
- o Unauthorized disruption / change of sensitive information in the system.

Medium - For the following risks:

- o The realization of the risk will lead to the immediate and direct shutdown of an insensitive system.

- o The realization of the risk may, in an uncertain manner, lead to the shutdown of a sensitive system.
- o Exposure of non-public inside information.
- o Unauthorized disruption / change of information that is not sensitive in the system in a way that will require a lot of effort in data recovery.

Low - For other serious risks.

Informative - For information provided.

6. Probability of realization - how to define the reasonableness of the risk:

Critical - A critical likelihood will be defined in a situation where it is found that the exposure has already been actually exercised (by a non-examining entity) or is available for immediate exploitation without the need for any preparation.

High - High probability will be defined in the following situations:

- o The risk can be realized by Social Engineering simply.
- o No technological knowledge is required or the required technological knowledge is not extensive.
- o Well-documented behavior.
- o The time required to realize the risk is small.
- o Ability to use mechanized tools.

Medium - Moderate likelihood will be defined in the following situations:

- o Information is available online.
- o Well-documented behavior.
- o The period of time required to realize the risk is long.

Low - Lower than moderate probability or in situations only theoretically there is a chance of exploiting the weakness.